

Звіт

до Лабораторної роботи №9

Узагальнення Generics в Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок використання узагальнених типів у Java, створення типобезпечних generic-класів і generic-методів, а також застосування обмежень параметрів типів.

У роботі необхідно реалізувати generic-клас, generic-метод, використати обмеження параметра типу та продемонструвати роботу generic-компонента у кількох різних конкретизаціях.

2. Варіант роботи

Варіант 5: медичні записи

Для обраного варіанта потрібно створити предметну модель медичних записів. У роботі реалізовано класи Patient, Visit і Prescription.

3. Опис предметної області

У лабораторній роботі розглядається предметна область медичних записів. У такій системі можуть існувати пацієнти, візити пацієнтів до лікарів і рецепти, які призначаються пацієнтам.

Пацієнт може мати різні медичні записи. Одні записи можуть описувати візит до лікаря, інші — призначений рецепт. Для збереження медичної сутності разом із додатковою інформацією використовується generic-клас `MedicalRecord<TEntity, TMeta>`.

4. Структура проєкту

Пакет	Призначення
model	Класи предметної області та generic-клас.
util	Утилітарний клас із generic-методом.
demo	Демонстраційний запуск програми.

5. Класи предметної області

5.1 Клас `Patient`

Клас `Patient` описує пацієнта.

Поля класу:

- `patientId` — ідентифікатор пацієнта;
- `fullName` — ПІБ пацієнта;
- `age` — вік пацієнта.

Клас містить конструктор, гетери та метод `toString()`.

5.2 Клас `Visit`

Клас `Visit` описує медичний візит пацієнта до лікаря.

Поля класу:

- `visitId` — ідентифікатор візиту;
- `patient` — пацієнт;

- visitDate — дата візиту;
- doctorName — ім'я лікаря.

Клас Visit реалізує інтерфейс Comparable<Visit>. Порівняння виконується за датою візиту. Пізніший візит вважається більшим.

5.3 Клас Prescription

Клас Prescription описує рецепт, який призначається пацієнту.

Поля класу:

- prescriptionId — ідентифікатор рецепта;
- patient — пацієнт;
- medicineName — назва препарату;
- doseMg — доза препарату в міліграмах;
- days — тривалість лікування в днях.

Клас Prescription реалізує інтерфейс Comparable<Prescription>. Порівняння виконується за кількістю днів лікування. Рецепт із більшою тривалістю вважається більшим.

6. Generic-клас MedicalRecord

У роботі реалізовано generic-клас:

MedicalRecord<TEntity, TMeta>

Цей клас має два параметри типу:

- TEntity — тип основного медичного об'єкта;
- TMeta — тип метаданих.

Основні поля generic-класу:

private final TEntity entity;

private final TMeta metadata;

Клас MedicalRecord є generic-компонентом, тому що він може працювати з різними типами медичних сутностей і різними типами метаданих без дублювання коду.

7. Приклади конкретизації generic-класу

У класі Main показано дві різні конкретизації generic-класу.

Перша конкретизація:

```
MedicalRecord<Visit, String> visitRecord =  
    new MedicalRecord<>(visit1, "Плановий візит");
```

У цьому випадку основним об'єктом є Visit, а метаданими є рядок String.

Друга конкретизація:

```
MedicalRecord<Prescription, Integer> prescriptionRecord =  
    new MedicalRecord<>(prescription1, 1);
```

У цьому випадку основним об'єктом є Prescription, а метаданими є число Integer.

Це показує, що один і той самий generic-клас може безпечно використовуватися для різних типів даних.

8. Generic-метод findMaximum

У роботі реалізовано generic-метод у класі MedicalUtils:

```
public static <T extends Comparable<? super T>> T findMaximum(List<T>  
items)
```

Метод приймає список елементів і повертає найбільший елемент. Якщо список порожній або дорівнює null, метод повертає null.

Метод використовується для:

- пошуку найпізнішого візиту;
- пошуку рецепта з найбільшою тривалістю лікування;
- пошуку найбільшого числа у списку Integer.

9. Обмеження параметра типу

У generic-методі використано обмеження:

```
<T extends Comparable<? super T>>
```

Це означає, що метод може працювати тільки з тими типами, які можна порівнювати.

Таке обмеження є змістовним, тому що метод `findMaximum()` повинен порівнювати елементи між собою. Без обмеження `Comparable` неможливо було б викликати метод `compareTo()`.

У даній моделі:

- `Visit` реалізує `Comparable<Visit>`;
- `Prescription` реалізує `Comparable<Prescription>`;
- `Integer` уже реалізує `Comparable<Integer>`.

Тому всі ці типи можуть бути використані у методі `findMaximum()`.

10. Чому використання `generics` є доречним

`Generics` у цій роботі використані не формально, а для покращення типобезпеки.

Без `generics` можна було б створити клас, який зберігає поля типу `Object`:

`Object entity;`

`Object metadata;`

Але такий підхід має недоліки:

- втрачається інформація про фактичний тип;
- потрібні явні приведення типів;
- частина помилок може з'явитися тільки під час виконання програми;
- код стає менш зрозумілим.

Generic-клас `MedicalRecord<TEntity, TMeta>` дозволяє уникнути цих проблем. Компілятор одразу знає, який тип має основний об'єкт і який тип мають метадані.

11. UML-діаграма

Для моделі було створено UML-діаграму у файлі `docs/uml.puml`. На діаграмі показано класи предметної області, generic-клас, generic-метод та зв'язки між класами.

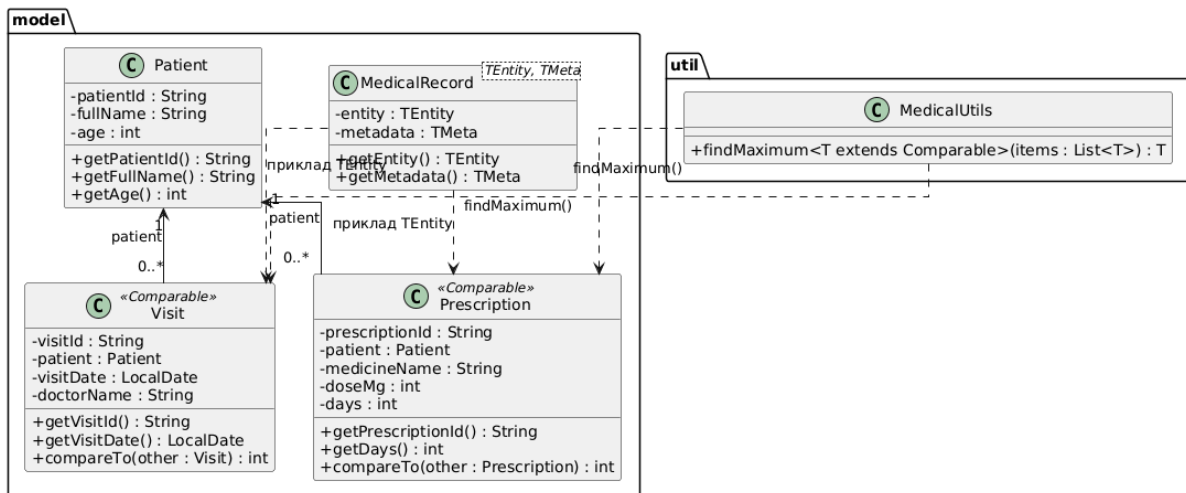


Рисунок 1 — UML-діаграма моделі медичних записів з використанням generics

12. Демонстрація роботи програми

У класі Main демонструється така послідовність дій:

1. Створення об'єкта Patient.
2. Створення двох об'єктів Visit.
3. Створення двох об'єктів Prescription.
4. Створення MedicalRecord<Visit, String>.
5. Створення MedicalRecord<Prescription, Integer>.
6. Виклик generic-методу findMaximum() для списку візитів.
7. Виклик generic-методу findMaximum() для списку рецептів.

Приклад результату роботи:

```

Generic-клас MedicalRecord
Медичний запис {об'єкт = Візит: VISIT-1, пацієнт: Колісниченко Артем, дата: 2026-04-10, лікар: Олена Володимирівна, метадані = Плановий візит}
Медичний запис {об'єкт = Рецепт: PRES-1, пацієнт: Колісниченко Артем, препарат: Парацетамол, доза: 500 мг, тривалість: 5 днів, метадані = 1}

Generic-метод findMaximum
Найпізніший візит:
Візит: VISIT-2, пацієнт: Колісниченко Артем, дата: 2026-05-15, лікар: Ігор Миколайович

Рецепт з найбільшою тривалістю:
Рецепт: PRES-2, пацієнт: Колісниченко Артем, препарат: Ібупрофен, доза: 400 мг, тривалість: 7 днів
  
```

Рисунок 2 — результат запуску програми

13. Тестування

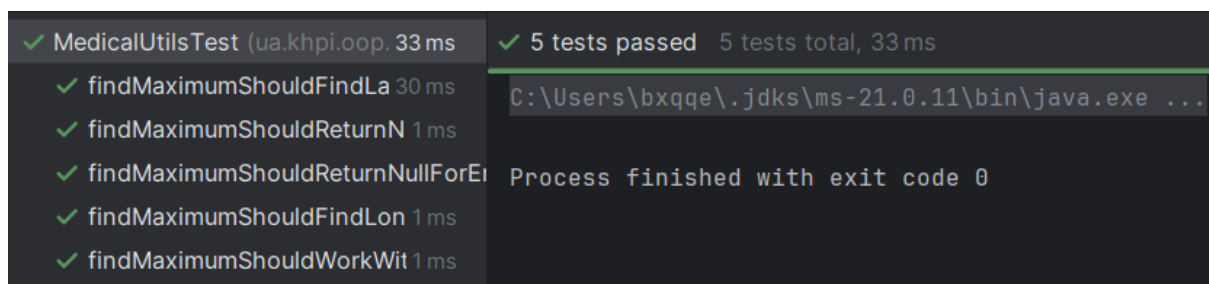
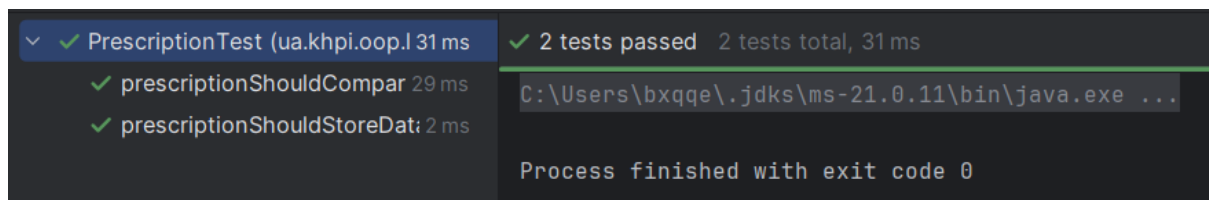
Для перевірки роботи програми були створені тести JUnit Jupiter.

Тестові класи:

- PatientTest;
- VisitTest;
- PrescriptionTest;
- MedicalRecordTest;
- MedicalUtilsTest.

Тести перевіряють:

- створення об'єкта Patient;
- створення об'єкта Visit;
- створення об'єкта Prescription;
- роботу compareTo() у класі Visit;
- роботу compareTo() у класі Prescription;
- роботу generic-класу MedicalRecord у двох конкретизаціях;
- роботу generic-методу findMaximum() з різними типами;
- повернення null для порожнього або null-списку.



Рисунки 3 та 4 — результати виконання тестів

14. Додаткове завдання

Для додаткового завдання можна переосмислити попередню лабораторну роботу №8, де розглядалася модель робочого простору проєкту.

У ній були класи:

- TaskCard;
- BugReport;
- ReviewNote;
- ProjectWorkspace.

У цій моделі можна було б узагальнити клас-контейнер. Наприклад, замість окремого зберігання різних списків можна запропонувати generic-сховище:

Repository<T>

Такий клас міг би містити список елементів одного типу:

List<T> items

І методи:

add(T item)

getAll()

find(...)

Тоді можна було б створити різні конкретизації:

Repository<TaskCard>

Repository<BugReport>

Repository<ReviewNote>

Таке узагальнення є доречним, якщо для різних типів об'єктів потрібна однакова логіка зберігання і пошуку. Але якщо кожен тип має зовсім різну логіку обробки, тоді generic-сховище може бути зайвим.

15. Висновок

У ході виконання лабораторної роботи було створено модель медичних записів для варіанта 5.

Було реалізовано класи предметної області: Patient, Visit і Prescription.

Для типобезпечного зберігання медичної сутності разом із метаданими було створено generic-клас `MedicalRecord<TEntity, TMeta>`.

Також було реалізовано generic-метод `findMaximum(List<T> items)` з обмеженням параметра типу `<T extends Comparable<? super T>>`.

Це обмеження є змістовним, тому що метод повинен порівнювати елементи списку між собою.

У класі `Main` було показано дві різні конкретизації generic-класу:

- `MedicalRecord<Visit, String>`;
- `MedicalRecord<Prescription, Integer>`.

Також було показано роботу generic-методу з різними типами: `Visit`, `Prescription` і `Integer`.

Отже, використання generics у цій роботі є доцільним, тому що воно дозволяє повторно використовувати один компонент для різних типів даних, зберігати типобезпеку і не використовувати небезпечні приведення типів.